

Introduction

C is one of the oldest and most widely used programming languages, but it's far from outdated. Despite being developed in the 1970s, C continues to be the backbone of many modern systems, including operating systems, embedded devices, and more.

Whether you're a beginner or looking to build a solid foundation in programming, understanding C is essential. In this introduction to C programming, we'll explore what is C language, why it's still relevant, and how learning it can help you for the career.

What is C Programming?

C is a powerful and flexible programming language used for a wide range of applications, from developing software to operating systems. It's known for its ability to interact directly with hardware, making it an excellent choice for tasks that require high performance and efficiency.

C allows programmers to write code that runs fast, while also offering features like structured programming and a rich set of functions. Even though it's considered a low-level language, C provides a solid foundation for learning other programming languages like C++, Java, and Python.

Key Features of C Language

Below are the main C programming features that make it popular and widely used:

- **Simple and Efficient:** C offers straightforward syntax and commands, making it easy to learn and use while maintaining high performance.
- **Structured Programming:** C language allows the breaking down of a program into functions, making the code easier to understand and manage.
- **Low-level Access:** C gives direct access to memory through pointers, allowing fine control over hardware resources.
- **Portability:** C programs can be run on different machines with minimal or no modification.
- **Rich Library Support:** C language has a vast collection of built-in functions and libraries for various tasks.
- **Modularity:** C enables code reusability by allowing functions and modules to be reused in different programs.
- **Dynamic Memory Allocation:** C programming language provides functions to allocate memory dynamically, giving control over memory management.
- **Fast Execution:** Programs written in C are compiled, making them faster compared to interpreted languages.

Why is C Language Still Popular?

C is still popular for several reasons, despite being one of the oldest programming languages:

1. High Performance

C is known for its speed. The language allows for direct memory manipulation, which makes it ideal for systems where performance is critical, like operating systems and embedded systems.

2. System-Level Programming

C is widely used in developing operating systems, drivers, and other low-level software. It gives programmers close control over the hardware, which is essential for system-level tasks.

3. Foundation for Other Languages

Many modern languages like C++, Java, and Python are either directly based on or influenced by C. Learning C provides a solid foundation for learning other programming languages.

4. Portability

C programs can be easily adapted to different machines and operating systems with minimal changes, making it a versatile choice for cross-platform development.

5. Widely Used in Industry

C is still the go-to language in industries like embedded systems, game development, and operating system development. It is also heavily used in compilers and network programming.

6. Community and Resources

With its long history, C has a vast community and a wealth of learning resources, libraries, and frameworks, making it easier for beginners to find support.

History of C

C language was developed in the early 1970s by Dennis Ritchie at Bell Labs. It was created as an improvement over the B language, which itself was a simplified version of BCPL.

Initially, C was designed to rewrite the Unix operating system, which was originally written in assembly language. Its ability to provide low-level access to memory and hardware, combined with high-level features like functions and structured programming, made it ideal for system programming.

Over time, C became widely adopted for a variety of applications, influencing the development of many modern languages.

Let's go through a timeline of the history of C language:

- **1960:** ALGOL introduced, which laid the groundwork for future languages like C.
- **1967:** Martin Richards developed BCPL, a precursor to C.
- **1970:** Ken Thompson created the B language, based on BCPL, for systems programming.
- **1972:** Dennis Ritchie developed C at Bell Labs, adding data types and structures to B.
- **1978:** The first edition of The C Programming Language by Kernighan and Ritchie was published.
- **1989:** ANSI C standard was established, providing a standardized version.
- **1999:** C99 introduced new features, including inline functions and variable-length arrays.
- **2011:** C11 added improved multithreading support and better Unicode handling.
- **2018:** C18 standard released with minor revisions.

Versions of C Language

These are the different versions of C:

Version	Year	Key Features
C89/ANSI C	1989	First standardized version by ANSI, ensuring consistency across compilers.
C99	1999	Added inline functions, variable-length arrays, and improved floating-point support.
C11	2011	Introduced multithreading support, anonymous structures, and atomic operations.
C17/C18	2018	Minor revision to C11, focused on bug fixes. Officially ISO/IEC 9899:2018.
C23	Expected 2024	Set to replace C17, introducing further updates to modernize the language

Uses and Applications of C

C programming is widely used across various domains due to its efficiency, low-level features, and portability:

- **Operating Systems:** Many popular operating systems like Unix, Linux, and parts of Windows are written in C due to its ability to interact directly with hardware.
- **Embedded Systems:** C is widely used in embedded programming, which includes microcontrollers, IoT devices, and automotive systems due to its memory efficiency.
- **Game Development:** Games usually require high performance, which C provides, making it a common choice for game engines and physics calculations.
- **Compilers:** C is used to develop compilers for other programming languages. Its versatility makes it ideal for building such system software.
- **Database Systems:** Database management systems like MySQL are written in C, where performance and efficient memory management are crucial.
- **Network Programming:** C is used for developing network protocols and communications systems due to its ability to handle low-level networking operations.
- **Graphics and GUI Development:** C plays a role in developing graphics libraries and GUIs, providing fast execution for rendering operations.

Basic Structure of C Program

Syntax

A C program consists of the following elements:

- **Headers:** These include necessary libraries using `#include`, such as `#include <stdio.h>`, which allows input/output functions.
- **Main Function:** Every C program must have a `main()` function, which is the entry point of the program.
- **Statements:** Inside `main()`, we write the program logic, typically as a series of function calls and operations.
- **Semicolons and Braces:** Semicolons (`;`) are used to end statements, and braces (`{ }`) define code blocks, ensuring the program's structure.

Example: "Hello, World!" Program

```
#include <stdio.h>
int main() {
    printf("Hello, World!\n");
}
```

Run Code

Output:

Hello, World!

Explanation:

- **`#include <stdio.h>`:** Includes the standard I/O library for `printf()`.
- **`int main()`:** Entry point of the program.
- **`printf("Hello, World!\n");`:** Prints "Hello, World!" to the console

Basics of C Language

Now that you know what is C programming and its key features & uses, let's move to learn the basics of C:

1. Data Types

C supports multiple data types to store different kinds of values:

Category	Data Types
----------	------------

Basic Data Types	int (for integers), float (for floating-point numbers), double (for precision floating-point), char (for characters)
Derived Data Types	Arrays, pointers, and functions
User-defined Types	struct (structure), union, and enum

2. Variables

Variables in C store data and need to be declared with a specific data type.

For example, `int a;` declares an integer variable `a`. Variables must be declared before they are used.

3. Operators

There are several operators in C to perform different operations on variables:

Category	Description
Arithmetic Operators	+ (Addition) - (Subtraction) * (Multiplication) / (Division) % (Modulus)
Relational Operators	== (Equal to) != (Not equal to) > (Greater than) < (Less than) >= (Greater than or equal to) <= (Less than or equal to)
Logical Operators	&& (Logical AND) (Logical OR) ! (Logical NOT)
Bitwise Operators	& (AND) (OR) ^ (XOR) ~ (NOT) << (Left Shift) >> (Right Shift)
Assignment Operators	= (Simple assignment) += (Add and assign) -= (Subtract and assign) *= (Multiply and assign) /= (Divide and assign) %= (Modulus and assign)

Unary Operators	++ (Increment) -- (Decrement)
Ternary Operator	? : (for conditional expressions)

4. Control Structures

Control structures are used to direct the flow of execution in a program:

- **Decision Making:** if, else, switch statements.
- **Loops:** for, while, and do-while loops for repeated execution of a block of code.
- **Break and Continue:** These statements alter the flow within loops.

5. Functions

C programs are modular and reusable through functions. A function is defined with a return type, a name, and parameters (optional).

For example:

```
int add(int a, int b) {
    return a + b;
}
```

Functions help break the code into smaller, manageable parts.

6. Pointers

Pointers are variables that store the memory address of another variable. They are an essential feature of C, allowing for:

- Dynamic memory allocation.
- Efficient array and string manipulation.
- Function pointers, passing by reference.

7. Arrays

Arrays in C are used to store multiple values of the same data type. They are defined by specifying the data type, followed by square brackets:

```
int arr[10]; // Array of 10 integers
```

8. Strings

Strings in C are arrays of characters terminated by a null character (`\0`).

They are declared as:

```
char str[20] = "Hello, World!";
```

9. Structures

Structures are user-defined data types that group different types of data together.

Example

```
struct student {
    int id;
    char name[50];
    float marks;
};
```

10. Dynamic Memory Allocation

C provides functions like `malloc()`, `calloc()`, `realloc()`, and `free()` to allocate and deallocate memory dynamically. This gives programmers control over the system's memory.

11. File Handling

C allows reading from and writing to files using functions like `fopen()`, `fclose()`, `fread()`, `fwrite()`, and more from the `<stdio.h>` library. File operations are important for saving data permanently.

12. Preprocessor Directives

C programs use preprocessor directives like `#include` and `#define` for including files and macros, which are processed before compilation.

13. Comments

Comments in C are ignored by the compiler and are used to make the code more readable:

- **Single-line Comment:** `// This is a comment`
- **Multi-line Comment:** `/* This is a multi-line comment */`

Career in C Programming

There are various job profiles for C programmers:

- **System Software Developer:** Builds and maintains operating systems and system-level software.
- **Embedded Systems Engineer:** Programs microcontrollers and embedded devices for IoT or automotive systems.
- **Firmware Developer:** Develops low-level software for hardware devices.
- **Game Developer:** Creates game engines and performance-heavy gaming applications.
- **Compiler Developer:** Builds and optimizes compilers for programming languages.
- **OS Developer:** Specializes in creating and improving operating systems.
- **Database Developer:** Works on database management systems like MySQL.
- **Network Engineer:** Develops networking protocols and communication systems.
- **C Developer:** Creates general-purpose applications in various industries.
- **IoT Developer:** Programs IoT systems for smart devices.
- **Robotics Engineer:** Develops control software for robotics systems.
- **Security Engineer:** Specializes in security algorithms and cryptographic systems using C.

Advantages of C

- **Fast and Efficient:** C provides high performance as it is a compiled language and works closely with hardware.
- **Portable:** C programs can run on different machines with minimal changes.
- **Rich Library Support:** Provides numerous built-in functions for various tasks.
- **Low-Level Programming:** Allows direct manipulation of memory and hardware.

Limitations of C

- **No Object-Oriented Programming:** C lacks OOP features like classes and inheritance.
- **Manual Memory Management:** Programmers must handle memory allocation and deallocation manually, which can lead to errors.
- **Lack of Error Handling:** C doesn't have built-in error-handling mechanisms.
- **No Garbage Collection:** Requires careful management of memory to prevent memory leaks.

Difference Between C and C++

Below is a comparison to understand the difference between C and C++ languages:

Feature	C	C++
Programming Paradigm	Procedural Programming	Supports both Procedural and Object-Oriented
Inheritance	Not supported	Supports inheritance through classes

Memory Management	Manual, using functions like malloc() and free()	Can use constructors, destructors, and smart pointers
Function Overloading	Not supported	Supported
Standard Library	Limited (basic functions)	Extensive (STL, classes, etc.)
Speed	Generally faster due to simplicity	Slightly slower due to added features like OOP

Full History of C Language (Origin, Founder, Timeline)

Introduction

C is one of the oldest and most influential programming languages, playing a foundational role in modern software development. Its origin in the early 1970s by Dennis Ritchie at Bell Labs marked a turning point, as C became the backbone for operating systems like Unix.

Knowing the history of C language is key to appreciating its simplicity, efficiency, and portability, which continue to shape today's programming world.

Here, let's discuss the C programming history, from its origins to its lasting impact on languages like [C++](#), [Java](#), and [Python](#).

C Programming Founder

The founder of C programming language is Dennis Ritchie, who developed it in 1972 at Bell Labs.

Origin of C Language

The C language has its roots in two earlier languages: BCPL (Basic Combined Programming Language) and B.

- BCPL, developed by Martin Richards in the 1960s, was designed for writing compilers.
- B, created by Ken Thompson in 1970, was a simplified version of BCPL used in system programming for Unix.

In the early 1970s, at Bell Labs, Dennis Ritchie improved upon B to create C, adding data types and structures to make it more powerful and flexible.

C quickly became the go-to language for system software, especially for writing the Unix operating system.

History of C (Timeline)

Below is the history of C in points with timeline:

Year	Event
1960s	BCPL (Basic Combined Programming Language) developed by Martin Richards, influencing later languages.
1970	B language created by Ken Thompson at Bell Labs, simplifying BCPL for system programming.
1972	Dennis Ritchie developed C at Bell Labs, adding data types and structures to B.
1973	Unix was rewritten in C, marking the rise of C in system programming.

1978	Kernighan & Ritchie published The C Programming Language, standardizing early C (K&R C).
1989	ANSI C (C89) was standardized, ensuring portability and consistency across platforms.
1999	C99 introduced features like inline functions and variable-length arrays.
2011	C11 brought enhancements such as multithreading support and improved Unicode handling.
2018	C17, a minor revision to C11, included bug fixes and slight improvements.
2023-24	C23 (expected 2024) set to replace C17, introducing further updates to modernize the language

Why Was C Developed?

In the late 1960s, Bell Labs was developing an operating system called Unix. Initially, Unix was written in assembly language, which was machine-dependent and difficult to maintain.

To solve this, Ken Thompson created the B language, a simplified version of BCPL, for Unix development. However, B lacked essential features for systems programming, like data types.

In 1972, Dennis Ritchie took B and enhanced it, introducing data types and structures, creating C. The goal was to create a language that offered the efficiency of assembly with the flexibility of a higher-level language, making Unix portable and easier to maintain.

After Dennis Ritchie developed C in 1972, it quickly became integral to rewriting Unix, making the operating system portable across machines. This adaptability led to widespread adoption of C, solidifying its reputation in system-level programming. By the late 1970s, C had grown beyond Bell Labs, influencing developers globally.

In 1989, C was standardized as ANSI C, ensuring code consistency across different platforms. Over the years, new versions like C99, C11, and C17 added features such as multithreading and dynamic memory handling.

Today, C remains an important language for operating systems, embedded systems, and high-performance applications. Its influence extends to modern languages like C++, Java, and Python, securing its legacy as one of the most important programming languages ever created.

Programming Languages Developed Before & Alongside C

Language	Year of Development	Developer	Key Features/Uses
Fortran	1957	John Backus and IBM	Primarily for scientific and engineering calculations.
LISP	1958	John McCarthy	Used in artificial intelligence and symbolic computation.
COBOL	1959	Grace Hopper and Committee	Designed for business and administrative systems.
ALGOL	1960	International Group	Major influence on later programming

			languages like Pascal, C.
BCPL	1967	Martin Richards	Basis for B language and foundational to C.
B	1970	Ken Thompson	Predecessor to C, used in early Unix systems.
C (Traditional)	1972	Dennis Ritchie	General-purpose, system programming, Unix OS.
K&R C	1978	Brian Kernighan, Dennis Ritchie	First official documentation and refinement of C.
ANSI C	1989	ANSI Committee	Standardized version for cross-platform compatibility.
C99	1999	Standardization Committee	Introduced new features like inline functions and better data types

C's Influence on Other Programming Languages

C has significantly influenced the development of many modern programming languages due to its simplicity, efficiency, and portability:

- **C++:** Developed as an extension of C, adding object-oriented programming features while retaining C's efficiency.
- **Java:** Inspired by C's syntax, Java uses C-like structures but with added memory management (garbage collection).
- **Python:** While a high-level language, Python adopts C's underlying concepts and is often implemented in C (CPython).
- **Go:** Incorporates C's simplicity and system-level programming focus with enhanced concurrency support.

Key Features of C

These are the key [features of C language](#):

- **Simplicity:** Straightforward syntax, easy to learn and understand.
- **Portability:** Code can be compiled on various platforms with minimal changes.
- **Modularity:** Functions help organize and reuse code efficiently.
- **Rich Library:** Provides numerous built-in functions for common tasks.
- **Low-level Memory Access:** Pointers allow direct manipulation of memory.
- **Efficiency:** Compiled into machine code, making programs fast.
- **Dynamic Memory Allocation:** Flexibility to allocate memory at runtime.
- **Structured Programming:** Code is organized into functions for clarity.
- **Recursion:** Functions can call themselves for complex problem-solving

Top Uses and Applications of C

These are the common [applications of C language](#)

- **Operating Systems:** Used in development of OS like Linux, Unix, Windows.
- **Embedded Systems:** Common in microcontrollers and IoT devices.
- **Game Development:** High-performance game engines rely on C.

- **System Programming:** C is widely used for developing drivers and kernels.
- **Database Systems:** C is the foundation for databases like MySQL.
- **Compilers:** C is used to write compilers for other languages.
- **Network Programming:** C is employed in building network protocols and applications.

Comments in C Language (All Types With Examples)

36 mins read

Last updated: 02 Mar 2026

9294 views

Share

6%

Table of Contents

Introduction

Comments in C are an essential part of writing clear and maintainable code. They allow programmers to explain the purpose, logic, and functionality of specific sections of code, which helps both the writer and others who may work on the code later.

Without proper comments, even a well-written program can become difficult to understand. By using C language comments effectively, you can provide valuable context for your code, making it easier to debug, modify, and collaborate with others.

Let's learn everything about comments in C code, including the types, examples, best practices, and more.

What Are Comments in C?

Comments in C language are notes you write in your code to explain what certain parts of the program do. They are ignored by the compiler when the program runs, but they help you or other developers understand the code later.

Think of comments in C as reminders or explanations that make your code easier to read and understand.

For example, if you write a complicated function, a comment can explain why you did it a certain way, so others don't have to guess. Comments make your code more organized and clearer.

Types of Comments in C Language

There are two types of comments in C:

1. Single-line Comments in C

The single-line comments in C language are used for brief notes or explanations and start with `//`. Everything after `//` on the same line is treated as a comment.

Syntax:

```
// This is a single-line comment
```

Example:

```
#include <stdio.h>
```

```
// This function adds two numbers and returns the sum
int add(int a, int b) {
    return a + b; // Perform the addition
}
```

```
int main() {
    int result = add(5, 3); // Calling the add function
    printf("Sum is: %d\n", result); // Display the result
    return 0;
}
```

Run Code

Output:

8

Explanation:

- **// This function adds two numbers and returns the sum**

This comment explains the purpose of the add function, indicating that the function is designed to take two numbers as inputs and return their sum.

- **return a + b; // Perform the addition**

The comment clarifies that this line performs the addition of the two input parameters (a and b) and returns the result. The comment helps readers understand that the returned value is the sum of a and b.

- **int result = add(5, 3); // Calling the add function**

The comment indicates that the add function is being called here with the arguments 5 and 3. The result of the addition is stored in the variable result.

- **printf("Sum is: %d\n", result); // Display the result**

The comment states that this line prints the result of the addition to the console using the printf function. The %d format specifier is used to display the integer value of result.

2. Multi-line Comments in C

These are used for longer explanations and are enclosed between /* and */. You can span comments across multiple lines.

Syntax:

```
/* This is a
   multi-line comment */
```

Example:

```
#include <stdio.h>
```

```
/*
```

This program calculates the area of a rectangle.

It takes the length and width as input and multiplies them to get the area.

```
*/
```

```
int main() {
```

```
    int length = 10;
```

```
    int width = 5;
```

```
    // Calculate area
```

```
    int area = length * width;
```

```
    /* Display the calculated area */
```

```
    printf("Area of the rectangle: %d\n", area);
```

```
    return 0;
}
```

Run Code

Output:

Area of the rectangle: 50

Explanation:

- Multiline Comment (`/* ... */`):

This comment block at the beginning provides a general description of what the program does.

It mentions that the program calculates the area of a rectangle by taking the length and width as inputs and then multiplying them to get the area.

Best Practices for Writing Comments in C Code

The following are some tips and best practices for using comments in C code:

- Keep comments clear and concise: Explain the "why" behind the code rather than the obvious "what."
- Comment complex logic: Add comments when the logic might be hard to understand later.
- Avoid over-commenting: Don't comment on every line; focus on key parts.
- Update comments with code changes: Ensure comments remain accurate as code evolves.
- Use consistent style: Stick to a uniform commenting style throughout the codebase.
- Avoid redundant comments: Comments that repeat the code don't add value.

How to Write Comments in C?

Below is how you should comment in [C programs](#) and best practices for placement:

1. Before Functions

Add comments to explain the purpose of a function, its input parameters, and its return values.

```
// This function adds two numbers and returns the result
```

```
int add(int a, int b) {
    return a + b;
}
```

2. Above Complex Logic

Use comments to describe complicated or non-obvious parts of the code.

```
// Check if the user is eligible based on multiple conditions
```

```
if (age > 18 && status == "active") {
    // Code to execute
}
```

3. To Clarify Unusual Code

Place comments near code that might not be intuitive.

```
// Using a bitwise operator to perform optimization
```

```
result = number & 0xFF;
```

4. End of Long Statements

For long code lines, add brief comments at the end to summarize the logic.

```
total = price * quantity + tax; // Calculate total cost including tax
```

By strategically placing comments, you can make your code more readable and understandable for yourself and others.

Examples of Comments in C

Below are C comment examples with brief explanations:

1. Program to Add Two Numbers

```
#include <stdio.h>
```

```
int main() {  
    int a = 5, b = 10;  
  
    // Adding two numbers  
    int sum = a + b;  
    printf("Sum of %d and %d is %d\n", a, b, sum); // Print result  
    return 0;  
}
```

Run Code

Output:

Sum of 5 and 10 is 15

Explanation:

- The single-line comment // Adding two numbers explains the logic of the addition.
- The comment at the print statement explains what the line outputs.

2. Program to Check if a Number is Prime

```
#include <stdio.h>
```

```
int main() {  
    int n = 29, flag = 1;  
  
    /* Checking if the number is divisible  
    by any number other than 1 and itself */  
    for (int i = 2; i <= n / 2; ++i) {  
        if (n % i == 0) {  
            flag = 0; // Number is not prime  
            break;  
        }  
    }  
}
```

```
// Output result
```

```
if (flag == 1)  
    printf("%d is a prime number.\n", n);  
else  
    printf("%d is not a prime number.\n", n);
```

```
return 0;
```

```
}
```

Run Code

Output:

29 is a prime number.

Explanation:

- The multi-line comment explains the logic behind checking divisibility.
- The single-line comment clarifies that flag = 0 means the number is not prime.
- Another single-line comment // Output result describes the output block.

3. Program to Calculate Factorial

```
#include <stdio.h>
```

```
int main() {  
    int n = 5;  
    int fact = 1;  
  
    // Calculating factorial of n  
    for (int i = 1; i <= n; ++i) {  
        fact *= i;  
    }  
  
    /* Displaying the factorial  
    of the given number */  
    printf("Factorial of %d is %d\n", n, fact);  
    return 0;  
}
```

Run Code

Output:

Factorial of 5 is 120

Explanation:

- The single-line comment // Calculating factorial of n explains the loop's purpose.
- The multi-line comment details that the next section prints the factorial result.

4. Program to Check Even or Odd

```
#include <stdio.h>
```

```
int main() {  
    int num = 4;  
    // Checking if the number is even or odd  
    if (num % 2 == 0) {  
        printf("%d is even.\n", num);  
    } else {  
        printf("%d is odd.\n", num);  
    }  
    return 0;  
}
```

Run Code

Output:

4 is even.

Explanation:

The comment // Checking if the number is even or odd explains the logic used to determine even or odd numbers.

Mistakes to Avoid While Writing Comments in C

Below are common mistakes to avoid while writing comments in C:

- **Over-commenting:** Adding comments to explain every single line, even when it's obvious.

- **Under-commenting:** Not explaining complex logic or sections of code that need clarification.
- **Outdated Comments:** Failing to update comments when the code changes, making them inaccurate.
- **Redundant Comments:** Repeating what the code already states clearly, adding no real value.
- **Writing Unclear Comments:** Using vague or confusing descriptions that don't help the reader understand the code.

Advantages of Comments in C Program

Below are some advantages of using comments in C programming:

- **Improves Code Readability:** Comments make code easier to understand for others or when you revisit your own code later.
- **Simplifies Debugging:** Clear comments help identify the purpose of each code block, making troubleshooting faster.
- **Facilitates Collaboration:** Teams working on the same code can better understand each other's logic and intent with well-written comments.
- **Maintains Code for Long-Term:** When code is revisited or updated, comments ensure the original logic is clear, reducing errors.
- **Explains Complex Logic:** For complicated algorithms or unusual methods, comments provide explanations that help prevent confusion.

Using Comments in C to Stop Code Execution

When debugging in C, you may want to stop a section of code from running temporarily without deleting it. You can do this by commenting out that part of the code.

This means that the code will be ignored by the compiler, but it remains in your program for future use. Once you're done debugging, you can simply remove the comments and the code will run again.

In example below, the `int sum` and `printf("Sum: %d\n", sum);` lines are commented out to prevent them from executing during debugging. You can later uncomment them when needed.

```
#include <stdio.h>
```

```
int main() {
```

```
    int a = 5, b = 10;
```

```
    // Temporarily disable this line during debugging
```

```
    // int sum = a + b;
```

```
    // printf("Sum: %d\n", sum); // Also disable this line
```

```
    printf("Code execution continues here...\n"); // This line will execute
```

```
    return 0;
```

```
}
```

Run Code

Output:

Code execution continues here...

Tokens in C Language (All Types With Examples)

Introduction

When learning [C programming](#), it's essential to understand how programs are structured. Tokens in C are the basic building blocks that make up the entire code. Every program consists of different types of C language tokens such as keywords, identifiers, constants, operators, and special symbols.

Learning these tokens will help you write efficient and clear code, as they are the foundation of every instruction in C. Let's talk about the various types of tokens and how they function within a C program.

What Are Tokens in C?

Tokens in C are the smallest units or building blocks of a C program. Every element of the code, like variables, functions, or symbols, is composed of tokens.

These include keywords, identifiers, constants, operators, and special symbols. Each token has a specific role in defining the structure and function of the program. Understanding how tokens work is essential to writing and understanding [C programs](#).

Types of Tokens in C Programming

The different types of tokens in C language include:

Token Type	Description
Keywords	Reserved words with predefined meanings (e.g., int, return).
Identifiers	Names used for variables, functions, and arrays (e.g., x, sum).
Constants	Fixed values that do not change (e.g., 5, 3.14).
Operators	Symbols that perform operations (e.g., +, -, *).
Strings	Sequence of characters enclosed in quotes (e.g., "Hello").
Special Symbols	Symbols used to define structure (e.g., {}, [], ;).

Below, we will discuss all the tokens in C language in detail with examples

Keywords

[Keywords in C](#) are reserved words that have special meanings and cannot be used as variable names, functions, or identifiers. These words are predefined by the language and help define the structure of the code.

There are 32 keywords in C:

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void

default	goto	sizeof	volatile
do	if	static	while

***Note:** The number of keywords in C can vary depending on the version. For example, ANSI C has 32 keywords, C11 has 44, and the latest C23 version includes around 54.*

Example of Keyword in C:

```
int main() {
    int num = 5; // 'int' and 'return' are keywords
    return 0;
}
```

Identifiers

Next in the list of tokens in C language are identifiers. The identifiers in C are the names used to identify variables, functions, arrays, or other entities in a program. They allow the programmer to refer to and manipulate data or functions.

Example:

```
int age;
float salary;
void myFunction() {
    // Function code
}
```

Rules for Naming Identifiers:

- Must start with a letter or underscore (_).
- Can only contain letters, digits, and underscores.
- Cannot use reserved keywords (e.g., int, return).
- Case-sensitive (e.g., age and Age are different).
- No special characters or spaces allowed.

Constants

Constants in C are fixed values that cannot be changed during the execution of a program. They are used to store data that remains the same throughout the program.

Example:

```
const int age = 25; // Integer constant
const float pi = 3.14; // Floating-point constant
const char grade = 'A'; // Character constant
```

Types of Constants:

- **Integer Constants:** Whole numbers like 5, -10.
- **Floating-point Constants:** Decimal numbers like 3.14, -0.001.
- **Character Constants:** Single characters like 'A', '9'.
- **String Constants:** Sequences of characters like "Hello".

Operators

Operators in C are symbols that perform operations on variables and values. They allow you to manipulate data and perform calculations.

Types of Operators:

Category	Description
Arithmetic Operators	+ (Addition)

	- (Subtraction) * (Multiplication) / (Division) % (Modulus)
Relational Operators	== (Equal to) != (Not equal to) > (Greater than) < (Less than) >= (Greater than or equal to) <= (Less than or equal to)
Logical Operators	&& (Logical AND) (Logical OR) ! (Logical NOT)
Bitwise Operators	& (AND) (OR) ^ (XOR) ~ (NOT) << (Left Shift) >> (Right Shift)
Assignment Operators	= (Simple assignment) += (Add and assign) -= (Subtract and assign) *= (Multiply and assign) /= (Divide and assign) %= (Modulus and assign)
Unary Operators	++ (Increment) -- (Decrement)
Ternary Operator	? : (for conditional expressions)

Example:

```
#include <stdio.h>
```

```
int main() {
    int a = 5, b = 3;

    // Arithmetic Operator
    int sum = a + b; // Adds a and b
    printf("Sum: %d\n", sum);

    // Relational Operator
```

```

if (a > b) {
    printf("%d is greater than %d\n", a, b); // '>' is a relational operator
}

// Logical Operator
if (a > 0 && b > 0) {
    printf("Both a and b are positive\n"); // '&&' is a logical operator
}

// Assignment Operator
a += b; // Adds b to a and assigns the result to a
printf("Updated a: %d\n", a);

return 0;
}

```

Run Code

Output:

Sum: 8

5 is greater than 3

Both a and b are positive

Updated a: 8

Explanation:

- $a + b$ is an arithmetic operator used to add the values of a and b and store the result in sum .
- $a > b$ is a relational operator used to compare whether a is greater than b .
- $\&\&$ is a logical operator used to check if both conditions are true.
- $a += b$ is an assignment operator used to add b to a and assign the updated value to a .

Special Symbols

Special symbols in C are characters that have specific meanings and play important roles in structuring and controlling the flow of a program.

Symbol	Description
Brackets []	Used to declare and access arrays.
Parentheses ()	Enclose function parameters and control expressions.
Braces {}	Define blocks of code for functions, loops, etc.
Comma ,	Separates variables, parameters, or elements in a list.
Colon :	Used in switch statements for labels.
Semicolon ;	Terminates statements.
Asterisk *	Used for pointers and multiplication.
Assignment =	Assigns values to variables.
Pre-processor #	Begins pre-processor directives like #include.

Period .	Accesses members of a structure.
Tilde ~	Performs bitwise negation.

Example:

```
#include <stdio.h>
```

```
int main() {
    int arr[5]; // [] brackets for array declaration
    int a = 10; // = is the assignment operator
    if (a > 5) {
        printf("a is greater than 5\n"); // {} braces for block of code
    }
    return 0; // () parentheses for function calls
}
```

Run Code

Output:

a is greater than 5

Explanation:

- [] square brackets are used to declare the array arr.
- = is an assignment operator used to assign the value 10 to variable a.
- {} braces are used to define a block of code inside the if condition.
- () parentheses are used with functions like main() and printf().

Example of Tokens in C Language

Let's go through an example of C language tokens:

```
#include <stdio.h> // Keyword and Special Symbol
```

```
int main() { // Keyword, Identifier, and Special Symbol
    int a = 10, b = 5; // Keyword, Identifier, Operator, and Special Symbol
    int sum = a + b; // Keyword, Identifier, Operator

    if (a > b) { // Keyword, Identifier, Relational Operator
        printf("a is greater than b\n"); // Keyword, String, Function call
    }

    printf("Sum: %d\n", sum); // Keyword, String, Identifier, Special Symbol
    return 0; // Keyword, Special Symbol
}
```

Tokens in Use:

- Keywords: int, if, return
- Identifiers: a, b, sum, main
- Constants: 10, 5
- Operators: +, >, =
- Special Symbols: (), {}, ;, #
- Strings: "a is greater than b\n"

Importance of Tokens in C Programming Language

- **Basic Building Blocks:** Tokens form the foundation of any C program, as every element of the code is a token.
- **Defines Structure:** Tokens like keywords, operators, and symbols define the logical structure of the program.
- **Compiler Understanding:** The compiler reads and interprets tokens to execute the code correctly.
- **Ensures Program Logic:** Proper use of tokens helps maintain the program's logic and flow.
- **Facilitates Readability:** Well-structured tokens improve code clarity and make it easier to understand and debug.

Programs Based on C Tokens

- [C Program to Add Two Integers](#)
- [C Program to Multiply Two Numbers](#)
- [C Program to Compute Quotient and Remainder](#)
- [C Program to Swap Two Numbers](#)
- [C Program to Check Whether a Number is Even or Odd](#)
- [C Program to Find the Largest Number Among Three Numbers](#)
- [C Program to Find the Roots of a Quadratic Equation](#)
- [C Program to Check Leap Year](#)

Keywords in C Language (Full List With Examples)

95 mins read

Last updated: 02 Mar 2026

63774 views

Share

8%

Table of Contents

Introduction

Keywords in C language are the building blocks that define the structure and functionality of a program. These keywords are reserved words that have special meanings and cannot be used as variable names or identifiers.

Learning how keywords in C like int, if, return, and for work is crucial because they control the flow of the program, define data types, and manage memory.

Let's discuss all types of keywords in C, how they function, and why they are essential for writing efficient and readable code.

What Are Keywords in C?

Keywords are reserved words that have predefined meanings and functions within the [C language](#). These words cannot be used as variable names, function names, or identifiers because they are part of the language's syntax and are already defined by the compiler.

Keywords help define the structure of the code, control program flow, manage memory, and specify data types.

For example, int, if, return, and for are commonly used C keywords. Each keyword has a specific purpose, and understanding how to use them correctly is essential for writing clear, efficient, and functional C programs.

List of All Keywords in C Language

The total number of keywords in C language is 32. The table below presents all 32 keywords:

Keyword	Meaning
auto	Declares automatic (local) variables.
break	Exits from loops or switch statements.
case	Defines a case in a switch statement.
char	Declares a character variable.
const	Declares a constant, immutable value.
continue	Skips the current iteration of a loop and proceeds to the next one.
default	Specifies the default case in a switch statement.
do	Executes a block of code at least once in a do-while loop.
double	Declares a double-precision floating-point variable.
else	Specifies the alternative branch in an if-else statement.
enum	Declares an enumeration, which is a list of named integer constants.
extern	Declares a global variable or function that is defined elsewhere.
float	Declares a floating-point variable.
for	Starts a loop that repeats until a condition is false.
goto	Transfers control to a labeled statement.
if	Specifies a condition to execute a block of code.
int	Declares an integer variable.
long	Declares a long integer variable.
register	Declares a variable that should be stored in a register for quick access.
return	Exits a function and optionally returns a value.
short	Declares a short integer variable.
signed	Declares a signed variable that can hold both positive and negative values.
sizeof	Returns the size, in bytes, of a data type or variable.
static	Declares variables with static lifetime or functions with limited visibility.
struct	Declares a structure, a collection of variables of different data types.

switch	Starts a multi-way branch based on the value of a variable.
typedef	Defines a new name for an existing data type.
union	Declares a union, a data type that can store different data types in the same memory location.
unsigned	Declares a variable that holds only non-negative values.
void	Specifies that a function does not return a value.
volatile	Indicates that a variable's value can be changed unexpectedly.
while	Starts a loop that repeats while a condition is true.

Note: The number of keywords in C may vary depending on the version. For example, ANSI C has 32 keywords, C11 has 44, and C23 is expected to introduce more, bringing the total to around 54.

Additional Keywords in C11

In C11, the following keywords were added:

- **_Alignas:** Specifies the alignment of a variable or type.
- **_Alignof:** Returns the alignment requirements of a type.
- **_Atomic:** Declares atomic types.
- **_Bool:** Boolean data type.
- **_Complex:** Declares complex numbers.
- **_Generic:** Provides generic programming capabilities.
- **_Imaginary:** Declares imaginary numbers.
- **_Noreturn:** Declares a function that does not return a value.
- **_Static_assert:** Provides compile-time assertions.
- **_Thread_local:** Declares thread-local storage.

Types of Keywords in C

We can classify C keywords as follows:

1. Data Type Keywords

Used to define the type of data that a variable holds.

Examples: int, char, float, double, void, _Bool, _Complex

2. Control Flow Keywords

Used to manage the flow of the program.

Examples: if, else, switch, case, for, while, do, goto

3. Storage Class Specifiers

Used to define the scope, visibility, and lifetime of variables.

Examples: auto, static, extern, register, _Thread_local

4. Type Qualifiers

Used to modify the properties of variables.

Examples: const, volatile, restrict, _Atomic

5. Function Related Keywords

Used to define or control functions.

Examples: return, _Noreturn, inline

6. Other Keywords

Special-purpose keywords used for structures, type definitions, memory alignment, and other advanced features.

Examples: typedef, struct, union, enum, _Alignas, _Static_assert

Keywords in C (With Examples)

Now that you have an idea about all keywords in C programming, let's discuss each in detail with example:

char

The char keyword in C is used to declare a character variable. A character is stored as a single byte in memory and holds a single character, typically using ASCII encoding.

Example

```
#include <stdio.h>
```

```
int main() {  
    char c = 'a'; // Declares a char variable  
    printf("%c", c); // Prints the value of the char variable  
    return 0;  
}
```

Run Code

Output:

a

int

The int keyword is used to declare integer variables, which store whole numbers without decimal points. It takes 2 or 4 bytes of memory depending on the system architecture.

Example:

```
#include <stdio.h>
```

```
int main() {  
    int num = 25; // Declare an integer variable  
    printf("%d", num); // Print the integer value  
    return 0;  
}
```

Run Code

Output:

25

float

The float keyword is used to declare floating-point variables, which store decimal values. It is typically used for more precise calculations than integers.

Example:

```
#include <stdio.h>
```

```
int main() {  
    float pi = 3.14; // Declare a float variable  
    printf("%.2f", pi); // Print the float value with two decimal points  
    return 0;  
}
```

Run Code

Output:

3.14

if

The if keyword is used to create a conditional statement that allows code to be executed only if a certain condition is true.

Example:

```
#include <stdio.h>
```

```
int main() {  
    int x = 10; // Declare a variable  
    if (x > 0) {  
        printf("Positive number");  
    }  
    return 0;  
}
```

Run Code

Output:

Positive number

else

The else keyword is used in conjunction with the if statement to specify a block of code that will execute if the if condition is false.

Example:

```
#include <stdio.h>
```

```
int main() {  
    int x = -5;  
    if (x > 0) {  
        printf("Positive");  
    } else {  
        printf("Not Positive");  
    }  
    return 0;  
}
```

Run Code

Output:

Not Positive

for

The for keyword is used to create a loop that runs a specific number of times. It contains an initialization, a condition, and an increment expression.

Example:

```
#include <stdio.h>
```

```
int main() {  
    for (int i = 0; i < 5; i++) {  
        printf("%d\n", i);  
    }  
}
```

```
    return 0;
}
```

Run Code

Output:

```
0
1
2
3
4
```

return

The return keyword is used to exit a function and optionally return a value to the calling function.

Example:

```
#include <stdio.h>
```

```
int main() {
    int result = sum(5, 10);
    printf("Sum = %d\n", result);
    return 0;
}
```

```
int sum(int a, int b) {
    return a + b; // Return sum to the main function
}
```

Run Code

Output:

```
Sum = 15
```

while

The while keyword is used to create a loop that continues executing as long as the condition is true.

Example:

```
#include <stdio.h>
```

```
int main() {
    int x = 0;
    while (x < 5) {
        printf("%d\n", x);
        x++;
    }
    return 0;
}
```

Run Code

Output:

```
0
1
2
```

3
4

switch

The switch keyword is used to select one of many code blocks to execute based on the value of a variable.

Example

```
#include <stdio.h>
```

```
int main() {  
    int x = 2;  
    switch (x) {  
        case 1: printf("One\n");  
            break;  
        case 2: printf("Two\n");  
            break;  
        default: printf("Other\n");  
    }  
    return 0;  
}
```

Run Code

Output:

Two

do

The do keyword is used with the while loop to execute a block of code at least once before checking the condition.

Example:

```
#include <stdio.h>
```

```
int main() {  
    int x = 0;  
    do {  
        printf("%d\n", x);  
        x++;  
    } while (x < 5);  
    return 0;  
}
```

Run Code

Output:

0
1
2
3
4

break

The break keyword is used to immediately exit a loop or switch statement when a specified condition is met.

Example:

```
#include <stdio.h>
```

```
int main() {  
    for (int i = 0; i < 5; i++) {  
        if (i == 3) break;  
        printf("%d\n", i);  
    }  
    return 0;  
}
```

Run Code

Output:

```
0  
1  
2
```

continue

The continue keyword skips the rest of the current iteration of a loop and moves on to the next iteration. It's useful when you want to bypass certain steps in a loop.

Example

```
#include <stdio.h>
```

```
int main() {  
    for (int i = 0; i < 5; i++) {  
        if (i == 2) continue;  
        printf("%d\n", i);  
    }  
    return 0;  
}
```

Run Code

Output:

```
0  
1  
3  
4
```

default

The default keyword is used in a switch statement to define the block of code that runs if none of the cases match the variable's value.

Example:

```
#include <stdio.h>
```

```
int main() {  
    int day = 5;  
    switch (day) {  
        case 1: printf("Monday\n"); break;  
        default: printf("Invalid day\n");  
    }  
}
```

```
    return 0;
}
```

Run Code

Output:

Invalid day

double

The double keyword is used to declare double-precision floating-point variables. It provides higher precision for decimal values compared to the float data type.

Example:

```
#include <stdio.h>
```

```
int main() {
    double pi = 3.141592653589793; // Declare a double variable
    printf("%lf\n", pi); // Print the double value
    return 0;
}
```

Run Code

Output:

3.141593

extern

The extern keyword is used to declare a global variable or function that is defined in another file or later in the same file. It allows multiple files to share the same variable or function.

Example:

```
#include <stdio.h>
```

```
extern int num; // External declaration (defined elsewhere)
```

```
int main() {
    printf("Number = %d\n", num); // Uses the external variable
    return 0;
}
```

```
int num = 10; // Definition of the external variable
```

Run Code

Output:

Number = 10

float

The float keyword is used to declare a floating-point variable, which stores decimal values with single precision.

Example:

```
#include <stdio.h>
```

```
int main() {
    float temperature = 36.5; // Declare a float variable
    printf("%.1f\n", temperature); // Print the float value
    return 0;
}
```

```
}
```

Run Code

Output:

36.5

goto

The goto keyword is used to transfer control to a labeled statement within the same function. It can lead to unstructured code and is generally discouraged.

Example:

```
#include <stdio.h>
```

```
int main() {  
    printf("Before the jump\n");  
    goto label; // Jump to the label  
    printf("This will be skipped\n");  
label:  
    printf("After the jump\n");  
    return 0;  
}
```

Run Code

Output:

Before the jump

After the jump

short

The short keyword is used to declare a short integer variable, which typically uses less memory than a regular int.

Example:

```
#include <stdio.h>
```

```
int main() {  
    short num = 100; // Declare a short integer  
    printf("%d\n", num); // Print the short integer  
    return 0;  
}
```

Run Code

Output:

100

int

The int keyword is used to declare integer variables that store whole numbers without decimal points.

Example:

```
#include <stdio.h>
```

```
int main() {  
    int number = 25; // Declare an integer variable  
    printf("%d\n", number); // Print the integer value  
    return 0;  
}
```

```
}
```

Run Code

Output:

25

long

The long keyword is used to declare a long integer variable, which can hold larger values than a regular int.

Example:

```
#include <stdio.h>
```

```
int main() {  
    long distance = 100000L; // Declare a long integer  
    printf("%ld\n", distance); // Print the long integer  
    return 0;  
}
```

Run Code

Output:

100000

signed

The signed keyword is used to declare an integer that can hold both positive and negative values. By default, int is signed in C.

Example:

```
#include <stdio.h>
```

```
int main() {  
    signed int balance = -500; // Declare a signed integer  
    printf("%d\n", balance); // Print the signed integer  
    return 0;  
}
```

Run Code

Output:

-500

unsigned:

The unsigned keyword is used to declare a variable that can only hold non-negative values. By default, int is signed, but unsigned removes the possibility of negative numbers, allowing the variable to store larger positive numbers.

Example:

```
#include <stdio.h>
```

```
int main() {  
    unsigned int positive_number = 100; // Declare an unsigned integer  
    printf("%u\n", positive_number); // Print the unsigned integer value  
    return 0;  
}
```

Run Code

Output:

100

void

The void keyword specifies that a function does not return any value. It is also used to indicate that a function does not take any parameters when defined.

Example:

```
#include <stdio.h>
```

```
void greet() { // Declare a void function
    printf("Hello, World!\n"); // Print greeting message
}
```

```
int main() {
    greet(); // Call the greet function
    return 0;
}
```

Run Code

Output:

Hello, World!

register

The register keyword suggests that the compiler store the variable in a register for faster access. This is just a suggestion, and the compiler may ignore it.

Example:

```
#include <stdio.h>
```

```
int main() {
    register int counter; // Suggest storing in register
    for (counter = 0; counter < 5; counter++) {
        printf("%d\n", counter);
    }
    return 0;
}
```

Run Code

Output:

0
1
2
3
4

sizeof

The sizeof keyword returns the size, in bytes, of a data type or variable. It is useful for finding out how much memory a data type or object takes.

Example:

```
#include <stdio.h>
```

```
int main() {
    printf("Size of int: %lu bytes\n", sizeof(int)); // Find size of int
}
```

```
    return 0;
}
```

Run Code

Output:

Size of int: 4 bytes

typedef

The typedef keyword is used to create an alias (or shorthand) for an existing data type, making the code more readable.

Example:

```
#include <stdio.h>
```

```
typedef unsigned int uint; // Create alias for unsigned int
```

```
int main() {
    uint x = 100; // Use the alias uint instead of unsigned int
    printf("%u\n", x); // Print the unsigned integer
    return 0;
}
```

Run Code

Output:

100

struct

The struct keyword is used to declare a structure, which is a user-defined data type that groups variables of different data types together.

Example:

```
#include <stdio.h>
```

```
struct Student {
    int id; // Declare an int variable
    char name[50]; // Declare a string variable
};
```

```
int main() {
    struct Student student1 = {1, "John"}; // Initialize structure
    printf("ID: %d, Name: %s\n", student1.id, student1.name);
    return 0;
}
```

Run Code

Output:

ID: 1, Name: John

union

The union keyword is used to declare a union, similar to a structure, but the members of a union share the same memory location. This means only one member can hold a value at a time.

Example:

```
#include <stdio.h>
```

```

union Data {
    int i;
    float f;
};

int main() {
    union Data data; // Declare a union
    data.i = 10; // Set integer value
    printf("i = %d\n", data.i);
    data.f = 22.5; // Set float value
    printf("f = %f\n", data.f); // Now f holds the value
    return 0;
}

```

Run Code

Output:

```

i = 10
f = 22.500000

```

volatile

The volatile keyword is used to tell the compiler that a variable's value can change at any time, even if it doesn't appear to be modified in the code. This is often used for hardware-related programming or multi-threaded programs.

Example

```
#include <stdio.h>
```

```
volatile int status = 1; // Declare a volatile variable
```

```

int main() {
    while (status) { // Use the volatile variable in a loop
        printf("System is running\n");
        status = 0; // Change the status to 0
    }
    return 0;
}

```

Run Code

Output:

```
System is running
```

const

The const keyword is used to define a constant variable, meaning that its value cannot be modified after initialization.

Example:

```
#include <stdio.h>
```

```

int main() {
    const int MAX = 100; // Declare a constant variable
    printf("Maximum value is %d\n", MAX);
    // MAX = 200; // This would generate an error
}

```

```
    return 0;
}
```

Run Code

Output:

Maximum value is 100

enum

The enum keyword is used to define an enumeration, which is a user-defined data type consisting of named integer constants.

Example:

```
#include <stdio.h>
```

```
enum Week {Sunday, Monday, Tuesday}; // Define an enumeration
```

```
int main() {
    enum Week today = Monday; // Assign an enum value
    printf("Today is %d\n", today); // Print the enum value (1 for Monday)
    return 0;
}
```

Run Code

Output:

Today is 1

auto

The auto keyword is used to declare automatic variables that are local to the function or block in which they are defined. By default, all local variables are automatic, so this keyword is rarely used.

Example:

```
#include <stdio.h>
```

```
int main() {
    auto int x = 10; // Local automatic variable
    printf("Value of x: %d\n", x);
    return 0;
}
```

Run Code

Output:

Value of x: 10

_Alignas

The _Alignas keyword is used to specify the alignment of a variable or a type in memory. It ensures that the variable is aligned to a specific boundary.

Example

```
#include <stdio.h>
```

```
int main() {
    _Alignas(double) int x; // Aligns x to the boundary required by double
    printf("Aligned integer value: %d\n", x);
    return 0;
}
```

```
}
```

Run Code

Output:

Aligned integer value: 0

_Alignof

The `_Alignof` keyword is used to determine the alignment requirement of a type in memory, returning the number of bytes required for alignment.

Example:

```
#include <stdio.h>
```

```
int main() {  
    printf("Alignment of int: %zu\n", _Alignof(int)); // Get alignment of int  
    return 0;  
}
```

Run Code

Output:

Alignment of int: 4

_Atomic

The `_Atomic` keyword is used to declare atomic types, which ensure that operations on variables are completed without being interrupted, useful in multi-threaded programming.

Example

```
#include <stdio.h>
```

```
#include <stdatomic.h>
```

```
int main() {  
    _Atomic int atomic_var = 0; // Declare an atomic variable  
    atomic_store(&atomic_var, 100); // Store value atomically  
    printf("Atomic variable: %d\n", atomic_var);  
    return 0;  
}
```

Run Code

Output:

Atomic variable: 100

_Bool

The `_Bool` keyword represents a Boolean data type in C11. It stores either 0 (false) or 1 (true).

Example:

```
#include <stdio.h>
```

```
int main() {  
    _Bool flag = 1; // True  
    if (flag) {  
        printf("Flag is true\n");  
    }  
    return 0;  
}
```

Run Code

Output:

Flag is true

_Complex

The _Complex keyword is used to declare complex numbers, which have both real and imaginary parts.

Example:

```
#include <stdio.h>
#include <complex.h>
```

```
int main() {
    double _Complex z = 1.0 + 2.0 * I; // Declare a complex number
    printf("Real part: %.1f, Imaginary part: %.1fn", creal(z), cimag(z));
    return 0;
}
```

Run Code

Output:

Real part: 1.0, Imaginary part: 2.0

_Generic

The _Generic keyword is used for generic programming by mapping different data types to specific expressions at compile time.

Example:

```
#include <stdio.h>

#define typeOf(x) _Generic((x), int: "int", float: "float", default: "unknown")
```

```
int main() {
    int a = 5;
    float b = 5.0;
    printf("Type of a: %s\n", typeOf(a)); // Type-checking using _Generic
    printf("Type of b: %s\n", typeOf(b));
    return 0;
}
```

Run Code

Output:

Type of a: int

Type of b: float

_Imaginary

The _Imaginary keyword is used to declare imaginary numbers. It stores only the imaginary part of a complex number.

Example:

```
#include <stdio.h>
#include <complex.h>
```

```
int main() {
    _Imaginary float img;
```

```

    img = 5.0i; // Assign imaginary value

    printf("Imaginary Value: %.2fi\n", img);

    return 0;
}

```

Run Code

Output:

Imaginary Value: 5.00i

Note: The `_Imaginary` keyword is part of the C99 standard, but most compilers like GCC do not fully support it. In modern C programming, `_Complex` is preferred for handling complex numbers.

`_Noreturn`

The `_Noreturn` keyword specifies that a function will not return to the calling function. It is typically used for functions that terminate the program (like `exit()`).

Example:

```

#include <stdio.h>
#include <stdlib.h>

```

```

_Noreturn void terminate() {
    printf("Terminating program\n");
    exit(0); // Exits the program without returning
}

```

```

int main() {
    terminate(); // Call the terminate function
}

```

Run Code

Output:

Terminating program

`_Static_assert`

The `_Static_assert` keyword is used to check conditions at compile time. It generates an error during compilation if the given condition is false.

Example:

```

#include <stdio.h>

```

```

int main() {
    _Static_assert(sizeof(int) == 4, "Int size is not 4 bytes"); // Compile-time check
    printf("Size of int is 4 bytes\n");
    return 0;
}

```

Run Code

Output:

Size of int is 4 bytes

`_Thread_local`

The `_Thread_local` keyword is used to declare thread-local storage. Each thread has its own instance of a `_Thread_local` variable, which is independent of other threads.

Example:

```
#include <stdio.h>
```

```
_Thread_local int counter = 0; // Declare a thread-local variable
```

```
int main() {  
    counter++;  
    printf("Counter: %d\n", counter);  
    return 0;  
}
```

Run Code

Output:

Counter: 1

Mistakes to Avoid When Using Keywords in C

Below are some common mistakes to avoid when using keywords in C programming:

1. Using Keywords as Identifiers

Do not name variables or functions using C keywords like `int`, `for`, or `if`.

Example (Incorrect):

```
int int = 10; (This will cause an error.)
```

2. Incorrect Use of return in Non-Returning Functions

Use `return` properly based on the function's return type.

Example:

void functions should not return a value.

3. Confusing == with = in Conditional Statements

Use `==` for comparison, not `=` (assignment).

Example (Incorrect):

```
if (x = 5) (This assigns instead of comparing).
```

4. Forgetting to Break in switch Statements

Always use `break` to avoid falling through to the next case unintentionally.

5. Mismatching Braces in if-else Statements

Ensure braces are balanced, especially in nested conditionals.

6. Misusing goto

Using `goto` can lead to difficult-to-debug, unstructured code. It's better to avoid it when possible.

7. Using extern Without Definition

Always ensure variables declared with `extern` are defined in some other file.

Importance of Keywords in C Language

- **Foundation of the Language:** Keywords are the basic building blocks of C. They define the syntax and [structure of C programs](#).
- **Reserved Words:** Keywords have predefined meanings, and the compiler interprets them to perform specific tasks.
- **Program Control:** Keywords like `if`, `for`, and `switch` control the flow of the program.
- **Data Declaration:** Keywords such as `int`, `char`, and `float` help in declaring variables and data types.

- **Error Prevention:** Proper use of keywords ensures code clarity and prevents ambiguity or unintended errors.

Data Types in C Language (With Examples)

33 mins read

Last updated: 23 Mar 2026

31011 views

Share

9%

Table of Contents

Introduction

Data types in C programming are essential as they define the type of data that a variable can hold, such as integers, characters, or floating-point numbers. Learning data types in C helps in managing memory efficiently and choosing the correct format for storing data.

C provides a variety of data types, including basic types like int, float, char, and more complex types like arrays, structures, and pointers.

We will discuss all types of data type in C language with examples, their sizes, and their practical applications in programming.

What Are Data Types in C?

Data types in [C program](#) help define the kind of data a variable can store, like numbers or letters. Each data type tells the compiler how much memory to allocate and how to interpret the data.

For instance, numbers without decimal points use a type like int, while numbers with decimal points use float or double. Characters, which store single letters or symbols, use the char type.

Data types are essential for organizing information efficiently and ensuring your program works as expected by managing memory properly.

Types of Data Type in C Language

C program data types are classified into three main categories:

1. Basic Data Types: These include fundamental types like int (for integers), char (for characters), float (for floating-point numbers), and double (for double-precision floating-point numbers).

2. Derived Data Types: These are derived from basic data types and include arrays, pointers, and functions.

3. User-Defined Data Types: These are created by the programmer and include structures (struct), unions (union), and enumerations (enum).

Basic Data Types	Derived Data Types	User-Defined Data Types
int	Arrays	struct
char	Pointers	union
float	Functions	enum
double		typedef

void		
------	--	--

Basic Data Types in C (With Examples)

Let's begin with the fundamental data types in C language:

1. int:

- Used to store integer values, which are whole numbers without a decimal point.
- Generally occupies 2 or 4 bytes, depending on the system.

Example:

```
#include <stdio.h>
```

```
int main(void) {  
    int age = 25;  
    printf("%d", age);  
    return 0;  
}
```

[Run Code](#)

Output:

25

2. char:

- Used to store single characters. Characters are enclosed in single quotes ('A').
- Occupies 1 byte of memory.

Example:

```
#include <stdio.h>
```

```
int main(void) {  
    char grade = 'A';  
    printf("%c", grade);  
    return 0;  
}
```

[Run Code](#)

Output:

A

3. float:

Used to store floating-point numbers (numbers with decimals). Generally occupies 4 bytes of memory.

Example:

```
#include <stdio.h>
```

```
int main(void) {  
    float temperature = 36.6f;  
    printf("%.1f", temperature);  
    return 0;  
}
```

[Run Code](#)

Output:

36.6

4. double:

Used to store double-precision floating-point numbers, offering more precision than float. Generally occupies 8 bytes of memory.

Example:

```
#include <stdio.h>
```

```
int main(void) {  
    double pi = 3.14159265359;  
    printf("%.10lf", pi);  
    return 0;  
}
```

Run Code

Output:

3.1415926536

Derived Data Types in C (With Examples)

Next, we will discuss the derived data types in C programming. These data types extend the functionality of basic types and help manage more complex data structures.

1. Arrays:

An array is a collection of elements of the same data type, stored in contiguous memory locations.

Example:

```
#include <stdio.h>
```

```
int main(void) {  
    int numbers[5] = {1, 2, 3, 4, 5};  
    printf("%d", numbers[0]);  
    return 0;  
}
```

Run Code

Output:

1

2. Pointers:

A pointer stores the memory address of another variable.

Example:

```
#include <stdio.h>
```

```
int main(void) {  
    int x = 10;  
    int *ptr = &x;  
    printf("%d", *ptr);  
    return 0;  
}
```

Run Code

Output:

10

3. Functions:

A function is a block of code that performs a specific task and can return a value.

Example:

```
#include <stdio.h>
```

```
int add(int a, int b) {  
    return a + b;  
}
```

```
int main(void) {  
    printf("%d", add(3, 5));  
    return 0;  
}
```

Run Code

Output:

8

User-Defined Data Types in C (With Examples)

The user-defined types in C language provide flexibility and enhance readability, especially when dealing with more complex data.

1. Structures (struct):

A structure is a user-defined data type that groups variables of different types under a single name.

Example:

```
#include <stdio.h>
```

```
struct Student {  
    int id;  
    char name[50];  
    float grade;  
};
```

```
int main(void) {  
    struct Student s1 = {1, "John", 95.5f};  
    printf("ID: %d, Name: %s, Grade: %.1f\n", s1.id, s1.name, s1.grade);  
    return 0;  
}
```

Run Code

Output:

ID: 1, Name: John, Grade: 95.5

2. Unions (union):

A union is similar to a structure, but members share the same memory location, meaning only one member can hold a value at a time.

Example:

```
#include <stdio.h>
```

```
union Data {  
    int i;
```

```

float f;
};

int main(void) {
    union Data data;

    data.i = 10;
    printf("i: %d\n", data.i);

    data.f = 20.5f;
    printf("f: %.2f\n", data.f);

    return 0;
}

```

Run Code

Output:

```

i: 10
f: 20.50

```

3. Enumerations (enum):

An enum is a data type consisting of named integer constants, improving code readability for sets of values.

Example:

```
#include <stdio.h>
```

```
enum Days { Sunday, Monday, Tuesday };
```

```

int main(void) {
    enum Days today = Monday;
    printf("Today is: %d\n", today);
    return 0;
}

```

Run Code

Output:

```
Today is: 1
```

Data Type Modifiers in C

We are now going to talk about Type Modifiers in C and how they affect the size or range of basic data types:

1. signed

By default, integer types are signed, meaning they can store both positive and negative values.

Example:

```
signed int num = -10; // Stores both positive and negative numbers.
```

2. unsigned

An unsigned variable can only hold non-negative values, allowing a larger positive range.

Example:

```
unsigned int num = 300; // Stores only positive values.
```

3. short

The short modifier is used to declare integer variables that typically use 2 bytes of memory, but the exact size may vary depending on the system or compiler.

Example:

```
short int num = 1000;
```

4. long

The long modifier is used to declare integer variables that typically use 4 bytes of memory, but the exact size may vary depending on the system or compiler.

Example:

```
long int bigNumber = 1000000;
```

Data Types in C With Size and Range

Different data types in C consume varying amounts of memory. For example, an int uses 2 or 4 bytes, a char uses 1 byte, a float uses 4 bytes, and a double uses 8 bytes. The size of these data types can vary depending on the system architecture (32-bit vs. 64-bit).

The table below shows the data types in C and their size along with range:

Data Type	Memory Size	Signed Range	Unsigned Range
char	1 byte	-128 to 127	0 to 255
short	2 bytes	-32,768 to 32,767	0 to 65,535
int	4 bytes	-2,147,483,648 to 2,147,483,647	0 to 4,294,967,295
long	4 bytes	-2,147,483,648 to 2,147,483,647	0 to 4,294,967,295
long long	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	0 to 18,446,744,073,709,551,615
float	4 bytes	3.4E-38 to 3.4E+38	N/A
double	8 bytes	1.7E-308 to 1.7E+308	N/A

sizeof Operator:

The sizeof operator is used to determine the size (in bytes) of any data type or variable.

Example:

```
#include <stdio.h>
```

```
int main(void) {  
    printf("Size of int: %lu bytes\n", sizeof(int));  
    return 0;  
}
```

Run Code

Output:

Size of int: 4 bytes

In this example, the sizeof operator is used to find the memory size of the int data type, which helps programmers manage memory efficiently.